

DevConnect

Random Video Chat Platform for Verified Engineers

GitHub-OAuth-Gated Video Networking App

Prepared for	Harsh Pandey
Project Type	Full-Stack Web/Mobile App (WebRTC + GitHub OAuth)
Document	Complete Step-by-Step Build Guide
Scope	Auth, Matching Engine, Video Calling, Moderation, Deployment

Table of Contents

1. Project Overview & Core Idea
2. System Architecture
3. Full Tech Stack
4. Data Storage Plan
5. Phase 1 — Environment & Project Setup
6. Phase 2 — GitHub OAuth Authentication
7. Phase 3 — Database Schema Design
8. Phase 4 — Backend API (FastAPI)
9. Phase 5 — Redis Matching Queue Engine
10. Phase 6 — WebRTC Peer Connection Setup
11. Phase 7 — Signaling Server (Socket.io / WebSockets)
12. Phase 8 — In-Call Features (Chat, Code Panel, Screen Share)
13. Phase 9 — Safety, Reporting & Moderation
14. Phase 10 — Frontend UI Build (Next.js)
15. Phase 11 — Testing Checklist
16. Phase 12 — Deployment
17. Timeline & Milestones
18. Appendix — Commands, Env Variables, Resources

1. Project Overview & Core Idea

DevConnect is a random video-chat platform exclusively for software engineers and developers. Unlike Monkey/Omegle (which allow anyone to sign up anonymously), this platform only allows login through GitHub OAuth — meaning every user is a real, identity-linked developer with a public coding history. This solves the anonymity/troll problem by design, and lets you build smart matching (by tech stack, role, or purpose) instead of pure randomness.

Core Value Proposition

- Only verified GitHub accounts can join — filters out bots and trolls automatically.
- Matching based on tech stack, role (student/SDE/freelancer) or purpose (job-search, mentorship, casual chat).
- In-call developer tools: code snippet sharing, screen share, visible GitHub profile card.
- Low moderation overhead compared to anonymous apps because identity is tied to a real GitHub account.

2. System Architecture

The system has 4 major runtime components that talk to each other. Video/audio itself flows directly peer-to-peer between the two users' browsers (WebRTC) — it never touches your server, which keeps costs and privacy risk low.

Component	Responsibility
Frontend (Next.js/React)	UI, GitHub login button, video call screen, chat overlay
Auth Service	GitHub OAuth handshake, JWT session issuing
Backend API (FastAPI)	User profiles, match history, reports, admin endpoints
Signaling Server (Socket.io)	Exchanges WebRTC SDP/ICE data between two matched users
Redis	Live matching queue + pub/sub for signaling
PostgreSQL	Persistent data: users, matches, reports, chat logs
TURN/STUN (coturn)	Helps peers connect through NAT/firewalls
S3 / Cloudflare R2	Stores avatars, report screenshots

Request Flow (Text Diagram)

User A clicks Start → Backend adds A to Redis queue → User B clicks Start → Backend finds A waiting, matches A+B → Signaling server tells both browsers to start WebRTC handshake → SDP/ICE exchanged via Socket.io → TURN/STUN helps establish direct P2P connection → Video/audio streams flow directly between A and B's browsers.

3. Full Tech Stack

Layer	Technology	Why
Frontend	Next.js (React) + TypeScript	SSR for auth pages, good WebRTC/browser API support
Styling	Tailwind CSS	Fast to build clean UI
Auth	NextAuth.js (GitHub provider)	Built-in GitHub OAuth flow, session handling
Backend API	FastAPI (Python)	Fast, async, good for GitHub API calls + WebSocket support
Signaling	Socket.io (Node) or FastAPI WebSockets	Real-time bidirectional messaging
Database	PostgreSQL	Relational data: users, matches, reports
Cache/Queue	Redis	Matching queue, pub/sub, session cache
Media	WebRTC + coturn (TURN/STUN)	P2P video/audio, NAT traversal
Storage	Cloudflare R2 / AWS S3	Avatars, report evidence
Hosting (MVP)	Railway / Render	Quick deploy, free/cheap tier for testing
Hosting (Scale)	AWS (EC2/ECS) + Load Balancer	Handles many concurrent signaling connections

4. Data Storage Plan

Data	Storage	Notes
User profile, GitHub data cache	PostgreSQL	Refreshed periodically, not real-time
Match history, friend connections	PostgreSQL	Relational, foreign keys to users
Reports/moderation logs	PostgreSQL	Permanent audit trail
Live matching queue	Redis	Ephemeral, cleared once matched
WebRTC signaling (SDP/ICE)	In-memory / Redis pub-sub	Only needed for a few seconds
Actual video/audio	Nowhere (P2P)	Never touches your server
Chat messages (optional history)	PostgreSQL or skip entirely	Your call — many apps don't persist this
Avatars, report screenshots	S3 / Cloudflare R2	Store URL reference in Postgres, not blobs

PHASE 1: Environment & Project Setup

Step 1.1 — Install Prerequisites

Install these on your machine before writing any code:

- Node.js (v18+) — for Next.js frontend and signaling server
- Python 3.11+ — for FastAPI backend
- PostgreSQL 15+ — local or via Docker
- Redis — local or via Docker
- Git + GitHub account (you'll also register an OAuth App)

Step 1.2 — Create Project Folder Structure

Set up a monorepo-style structure so everything lives together:

```
devconnect/  
├── frontend/ # Next.js app  
├── backend/ # FastAPI app  
├── signaling-server/ # Socket.io server  
├── docker-compose.yml # Postgres + Redis + coturn  
└── README.md
```

Step 1.3 — Docker Compose for Local Services

Instead of installing Postgres/Redis manually, run them in Docker:

```
# docker-compose.yml  
version: '3.8'  
services:  
  postgres:  
    image: postgres:15  
    environment:  
      POSTGRES_USER: devconnect  
      POSTGRES_PASSWORD: devconnect123  
      POSTGRES_DB: devconnect  
    ports:  
      - "5432:5432"  
  redis:  
    image: redis:7  
    ports:  
      - "6379:6379"  
  coturn:  
    image: coturn/coturn  
    ports:  
      - "3478:3478/udp"  
      - "3478:3478/tcp"
```

Step 1.4 — Start Services

```
docker-compose up -d
```

PHASE 2: GitHub OAuth Authentication

Step 2.1 — Register a GitHub OAuth App

Go to GitHub → Settings → Developer Settings → OAuth Apps → New OAuth App.

Fill in: Application name (DevConnect), Homepage URL (<http://localhost:3000>), Authorization callback URL (<http://localhost:3000/api/auth/callback/github>).

After creating, copy the Client ID and generate a Client Secret. Save both securely.

Step 2.2 — Install NextAuth.js in Frontend

```
cd frontend
npm install next-auth
```

Step 2.3 — Configure GitHub Provider

Create the NextAuth config file with the GitHub provider:

```
// frontend/pages/api/auth/[...nextauth].ts
import NextAuth from "next-auth"
import GitHubProvider from "next-auth/providers/github"

export default NextAuth({
  providers: [
    GitHubProvider({
      clientId: process.env.GITHUB_ID,
      clientSecret: process.env.GITHUB_SECRET,
      authorization: { params: { scope: "read:user user:email" } }
    })
  ],
  callbacks: {
    async jwt({ token, account, profile }) {
      if (account) {
        token.accessToken = account.access_token
        token.githubUsername = profile?.login
      }
      return token
    },
    async session({ session, token }) {
      session.accessToken = token.accessToken
      session.githubUsername = token.githubUsername
      return session
    }
  }
})
```

Step 2.4 — Add Environment Variables

```
# frontend/.env.local
GITHUB_ID=your_client_id
GITHUB_SECRET=your_client_secret
NEXTAUTH_SECRET=generate_a_random_string
NEXTAUTH_URL=http://localhost:3000
```

Step 2.5 — Add Login Button

```
// components/LoginButton.tsx
import { signIn, signOut, useSession } from "next-auth/react"

export default function LoginButton() {
  const { data: session } = useSession()
  if (session) return <button onClick={() => signOut()}>Sign out</button>
  return <button onClick={() => signIn("github")}>Sign in with GitHub</button>
}
```

Step 2.6 — Sync User to Backend on First Login

When a user logs in for the first time, call your FastAPI backend to create/update their profile row using the GitHub access token, so their repos/languages get cached in Postgres.

Note: Never expose the raw GitHub access token to the frontend beyond what's needed — store it encrypted server-side if you need to make repeated GitHub API calls.

PHASE 3: Database Schema Design

Step 3.1 — Users Table

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  github_id BIGINT UNIQUE NOT NULL,  
  username VARCHAR(100) NOT NULL,  
  avatar_url TEXT,  
  bio TEXT,  
  top_languages JSONB, -- e.g. ["Python", "TypeScript"]  
  role VARCHAR(50), -- student / SDE / freelancer  
  looking_for VARCHAR(50), -- job / collab / mentorship / chat  
  access_token_enc TEXT, -- encrypted GitHub token  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Step 3.2 — Matches Table

```
CREATE TABLE matches (  
  id SERIAL PRIMARY KEY,  
  user_a_id INT REFERENCES users(id),  
  user_b_id INT REFERENCES users(id),  
  match_mode VARCHAR(30), -- random / tech_stack / role / purpose  
  started_at TIMESTAMP DEFAULT NOW(),  
  ended_at TIMESTAMP,  
  duration_seconds INT  
);
```

Step 3.3 — Friends / Connections Table

```
CREATE TABLE connections (  
  id SERIAL PRIMARY KEY,  
  user_id INT REFERENCES users(id),  
  friend_id INT REFERENCES users(id),  
  connected_at TIMESTAMP DEFAULT NOW()  
);
```

Step 3.4 — Reports Table

```
CREATE TABLE reports (  
  id SERIAL PRIMARY KEY,  
  reporter_id INT REFERENCES users(id),  
  reported_id INT REFERENCES users(id),  
  reason VARCHAR(100),  
  details TEXT,  
  evidence_url TEXT,  
  status VARCHAR(30) DEFAULT 'pending',  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Step 3.5 — Run Migrations

Use Alembic (FastAPI's standard migration tool) to version-control schema changes:

```
pip install alembic
alembic init migrations
alembic revision --autogenerate -m "initial schema"
alembic upgrade head
```

PHASE 4: Backend API (FastAPI)

Step 4.1 — Initialize FastAPI Project

```
cd backend
python -m venv venv
source venv/bin/activate
pip install fastapi uvicorn sqlalchemy psycpg2-binary redis python-jose alembic
uvicorn main:app --reload
```

Step 4.2 — Basic App Structure

```
backend/
├── main.py
├── models.py # SQLAlchemy models
├── database.py # DB connection
├── routes/
│   ├── users.py
│   ├── matching.py
│   ├── reports.py
└── requirements.txt
```

Step 4.3 — Create User Endpoint (called after GitHub login)

```
# routes/users.py
from fastapi import APIRouter
router = APIRouter()

@router.post("/users/sync")
def sync_user(github_id: int, username: str, avatar_url: str, bio: str):
    # upsert into Postgres: create if not exists, else update cached fields
    ...
    return {"status": "synced"}
```

Step 4.4 — JWT Session Verification

Every protected API call should verify the NextAuth JWT so the backend knows which user is calling.

Note: Share the NEXTAUTH_SECRET between frontend and backend so the backend can verify the token signature.

PHASE 5: Redis Matching Queue Engine

Step 5.1 — Queue Design

When a user clicks Start, push their user ID + filter tags into a Redis list/set. When a second compatible user clicks Start, pop the first match and pair them.

Step 5.2 — Basic Random Matching (Python + Redis)

```
import redis
r = redis.Redis(host='localhost', port=6379, db=0)

def join_queue(user_id):
    # try to find a waiting partner
    partner = r.lpop("waiting_queue")
    if partner:
        return pair_users(partner.decode(), user_id)
    else:
        r.rpush("waiting_queue", user_id)
    return None

def pair_users(user_a, user_b):
    room_id = f"room_{user_a}_{user_b}"
    r.set(f"user:{user_a}:room", room_id)
    r.set(f"user:{user_b}:room", room_id)
    return room_id
```

Step 5.3 — Tag-Based Matching (Tech Stack / Role)

Use separate Redis lists per tag, e.g. waiting_queue:python, waiting_queue:react, so users only match with others sharing that filter.

```
def join_tagged_queue(user_id, tag):
    key = f"waiting_queue:{tag}"
    partner = r.lpop(key)
    if partner:
        return pair_users(partner.decode(), user_id)
    r.rpush(key, user_id)
    return None
```

Step 5.4 — Handle Skip/Next

On skip: remove user from current room, push them back into the queue immediately, and notify their previous partner that the call ended so they can also re-queue.

PHASE 6: WebRTC Peer Connection Setup

Step 6.1 — Get Local Camera/Mic Access

```
const stream = await navigator.mediaDevices.getUserMedia({
  video: true,
  audio: true
});
localVideoRef.current.srcObject = stream;
```

Step 6.2 — Create RTCPeerConnection

```
const config = {
  iceServers: [
    { urls: "stun:stun.l.google.com:19302" },
    { urls: "turn:your-coturn-server:3478", username: "user", credential: "pass" }
  ]
};
const pc = new RTCPeerConnection(config);
stream.getTracks().forEach(track => pc.addTrack(track, stream));

pc.ontrack = (event) => {
  remoteVideoRef.current.srcObject = event.streams[0];
};
```

Step 6.3 — Create and Send Offer

```
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
socket.emit("webrtc-offer", { roomId, offer });
```

Step 6.4 — Handle Answer and ICE Candidates

```
socket.on("webrtc-answer", async ({ answer }) => {
  await pc.setRemoteDescription(new RTCSessionDescription(answer));
});

pc.onicecandidate = (event) => {
  if (event.candidate) {
    socket.emit("ice-candidate", { roomId, candidate: event.candidate });
  }
};

socket.on("ice-candidate", async ({ candidate }) => {
  await pc.addIceCandidate(new RTCIceCandidate(candidate));
});
```

PHASE 7: Signaling Server (Socket.io)

Step 7.1 — Initialize Signaling Server

```
cd signaling-server
npm init -y
npm install socket.io express redis
```

Step 7.2 — Basic Server Setup

```
// index.js
const { Server } = require("socket.io");
const io = new Server(4000, { cors: { origin: "*" } });

io.on("connection", (socket) => {
  socket.on("join-room", (roomId) => {
    socket.join(roomId);
  });

  socket.on("webrtc-offer", ({ roomId, offer }) => {
    socket.to(roomId).emit("webrtc-offer", { offer });
  });

  socket.on("webrtc-answer", ({ roomId, answer }) => {
    socket.to(roomId).emit("webrtc-answer", { answer });
  });

  socket.on("ice-candidate", ({ roomId, candidate }) => {
    socket.to(roomId).emit("ice-candidate", { candidate });
  });

  socket.on("disconnect", () => {
    // notify room partner, trigger re-queue logic
  });
});
```

Step 7.3 — Connect Signaling Server to Redis Match Events

When the FastAPI backend pairs two users in Redis, publish a message so the signaling server tells both sockets to join the same room and start the WebRTC handshake.

PHASE 8: In-Call Features

Step 8.1 — Text Chat Overlay

Use the same Socket.io connection to send small text messages within the room, rendered as an overlay on top of the video.

```
socket.emit("chat-message", { roomId, text });
socket.on("chat-message", ({ text, from }) => {
  // append to chat UI
});
```

Step 8.2 — Code Snippet Panel

Add a collapsible side panel with a simple code editor (Monaco or CodeMirror) so both users can view/edit a shared snippet live — sync changes over the same socket channel.

Step 8.3 — Screen Share

```
const screenStream = await navigator.mediaDevices.getDisplayMedia({ video: true });
const screenTrack = screenStream.getVideoTracks()[0];
const sender = pc.getSenders().find(s => s.track.kind === "video");
sender.replaceTrack(screenTrack);
```

Step 8.4 — GitHub Profile Card During Call

Fetch the matched partner's cached GitHub data (username, top languages, pinned repos) from your backend and display it as a small card next to the video.

PHASE 9: Safety, Reporting & Moderation

Step 9.1 — Report Button

Always-visible button during calls. On click, open a small form: reason dropdown (harassment, inappropriate content, spam, other) + optional details.

```
POST /reports
{
  "reported_id": 42,
  "reason": "inappropriate_content",
  "details": "..."
```

Step 9.2 — Auto-Flag Repeat Offenders

If a user accumulates more than N reports within a time window, auto-suspend their account pending review, and log the action.

Step 9.3 — Block List

Maintain a blocked_users table/array so blocked users are never matched with each other again.

Step 9.4 — Basic Content Moderation

Since login is GitHub-gated, anonymous abuse risk is already lower. Still add a text-chat profanity filter, and consider optional frame-sampling with a moderation API if video abuse becomes a problem at scale.

Note: Full real-time video moderation (like commercial Monkey/Omegle) is expensive — for an MVP, identity-gating + reporting + fast admin review is usually enough.

PHASE 10: Frontend UI Build (Next.js)

Step 10.1 — Pages to Build

- / — Landing page with 'Sign in with GitHub' button
- /dashboard — Profile summary, match mode selector, Start button
- /call/[roomId] — Video call screen with chat, code panel, report button
- /history — Past matches and connections
- /admin — Moderation dashboard (protected route)

Step 10.2 — Call Screen Layout

Two video tiles (self + remote), floating controls (mute, camera, skip, report), collapsible chat + code panel on the side.

Step 10.3 — State Management

Use React Context or Zustand to manage call state (connecting, connected, ended) across components cleanly.

PHASE 11: Testing Checklist

- GitHub OAuth login/logout works and correctly creates/updates the user row.
- Two browser tabs (or two devices) can be matched and see each other's video.
- Skip button correctly re-queues both users.
- Tag-based matching only pairs users sharing the selected tag.
- Chat messages appear on both sides in real time.
- Report flow creates a row in the reports table and is visible in admin panel.
- Reconnection handling: if one user's network drops, the other is notified and re-queued.
- TURN server works when both users are behind restrictive NATs (test on mobile data).

PHASE 12: Deployment

Step 12.1 — Deploy Frontend

Deploy the Next.js app to Vercel (easiest) — connect your GitHub repo, set the same environment variables (GITHUB_ID, GITHUB_SECRET, NEXTAUTH_SECRET, NEXTAUTH_URL with prod domain).

Step 12.2 — Deploy Backend + Signaling Server

Deploy FastAPI and the Socket.io server to Railway or Render for MVP. Use managed Postgres and Redis add-ons from the same platform to reduce latency.

Step 12.3 — Deploy coturn (TURN server)

Run coturn on a small VPS (e.g. DigitalOcean droplet) since managed platforms rarely host it well. Open UDP/TCP ports 3478 and a relay port range.

Step 12.4 — Update GitHub OAuth App Callback URL

Add your production domain's callback URL to the GitHub OAuth App settings (e.g. <https://devconnect.app/api/auth/callback/github>).

Step 12.5 — Set Up Monitoring

Add basic uptime monitoring (UptimeRobot) and error tracking (Sentry) for both frontend and backend.

17. Timeline & Milestones

Week	Milestone
Week 1	Environment setup, GitHub OAuth login working end-to-end
Week 2	Database schema + backend API (user sync, profile endpoints)
Week 3	Redis matching queue (random matching) + basic 1-to-1 WebRTC call
Week 4	Signaling server complete, skip/next logic, tag-based matching
Week 5	In-call features: chat overlay, code snippet panel, screen share
Week 6	Reporting system, block list, admin moderation panel
Week 7	Frontend polish, match history, connections page
Week 8	Testing, TURN server setup, deployment, monitoring

This is a solo-builder estimate assuming part-time work alongside coursework. It can compress to 3-4 weeks if working full-time, or if some phases (e.g. moderation, admin panel) are simplified for a first MVP demo.

18. Appendix — Commands, Env Variables, Resources

Environment Variables Checklist

```
# Frontend (.env.local)
GITHUB_ID=
GITHUB_SECRET=
NEXTAUTH_SECRET=
NEXTAUTH_URL=

# Backend (.env)
DATABASE_URL=postgresql://devconnect:devconnect123@localhost:5432/devconnect
REDIS_URL=redis://localhost:6379
JWT_SECRET= # must match NEXTAUTH_SECRET
GITHUB_CLIENT_SECRET=

# Signaling Server (.env)
REDIS_URL=redis://localhost:6379
PORT=4000
```

Useful Commands Reference

```
# Start all local services
docker-compose up -d

# Backend
uvicorn main:app --reload --port 8000

# Frontend
npm run dev

# Signaling server
node index.js

# DB migration
alembic revision --autogenerate -m "message"
alembic upgrade head
```

Key Resources

WebRTC basics: webrtc.org/getting-started

NextAuth GitHub provider docs: next-auth.js.org/providers/github

coturn setup guide: github.com/coturn/coturn

Socket.io docs: socket.io/docs